



Simulation des Energiebedarfs eines E-Bikes anhand von GPS-Streckendaten

Abschlussprojekt 2026

Programmieren I

Autor(en)

Matrikelnummer

Kohorte

Gruppe

Lektor

Thea Welk, Emil Kupfer, Franciszek Wozniak

12309793, 52506964, 52507104

Abschlussprojekt 2026

B

Peter Kandolf

Inhaltsverzeichnis

Zusammenfassung	1
1 Einleitung	2
1.1 Beitrag des Projekts	2
1.2 Dokumentation im Bericht	2
2 Funktionsweise des Programms im Überblick	3
2.1 Grundidee	3
2.2 Ablauf in neun Schritten	3
2.3 Zusammenspiel der Bausteine	4
2.4 Die zwei Konfigurationsdateien	4
2.5 Ergebnis in einem Satz	4
3 Anforderungen	5
3.1 Implementierte Kernfunktionen	5
3.2 Umgesetzte Erweiterungen	5
3.3 Technische Randbedingungen	6
4 Projektstruktur	7
4.1 Übersicht der Komponenten	7
4.2 Zentrale Klassen und Funktionen	8
4.3 Dateien und Ausgaben	9
5 Softwarearchitektur und OOP	10
5.1 Objektorientierte Struktur	10
5.2 Klassendiagramm	10
5.3 Daten- und Kontrolltrennung	10
5.4 Ein gemeinsamer Service-Layer für CLI und Web-App	11
6 GPS- und Kinematikberechnung	12
6.1 Zeit und Distanz	12
6.2 Steigung, Kurs und Kompassrichtung	12
6.3 Beschleunigung	13
6.4 Wichtige DataFrame-Spalten	13

7	Geschwindigkeitsglättung	14
7.1	Konfigurationsparameter	14
7.2	Ablauf der Glättung	15
7.3	Mathematische Beschreibung	16
7.4	Aktive und passive Spalten	16
8	Physikalisches E-Bike-Modell	17
8.1	Einzelkräfte	17
8.2	Gesamtkraft und Leistung	18
8.3	Parameter	18
9	Motor- und Akkumodelle	20
9.1	Motor	20
9.2	Abstrakte Akku-Schnittstelle	20
9.3	Gemeinsame Akku-Funktionalität	20
9.4	LiPo und NMC	21
9.5	Wichtige Attribute und Parameter	21
9.6	Bremswiderstand und Rekuperation	22
10	Simulationsablauf	23
10.1	Aktivitätsdiagramm der Simulation	23
10.2	Warum bricht die Simulation bei leerem Akku ab, statt weiterzurechnen?	23
10.3	Datenfluss	26
10.4	Ablauf in <code>main.py</code>	26
10.5	Übergebene Daten	26
11	Visualisierung und Ausgaben	27
11.1	HTML-Karten: wie der Streckenplot funktioniert	27
11.2	PNG-Diagramme	27
11.3	Logdatei und Verzeichnisse	28
12	Reverse Geocoding, Wetterdaten und Web-Anwendung	31
12.1	Reverse Geocoding: von Koordinaten zu Adressen	31
12.2	Wetterdaten: woher die Werte kommen	31
12.3	Statische GIS-Karten mit GeoPandas	32
12.4	Web-Anwendung und automatische Parameterstudien	32
13	Tests und Validierung	34
13.1	Ausgeführter Testbefehl	34
13.2	Ergebnis	34
13.3	Interpretation	34

14 Ergebnisse	36
14.1 Streckenkennwerte	36
14.2 Vergleich der Glättung	37
14.3 Interpretation	37
15 Grenzen und Ausblick	39
15.1 Aktuell implementiert	39
15.2 Modellannahmen	39
15.3 Moegliche Erweiterungen	39
15.4 Einordnung	40
16 Fazit	41

Abbildungsverzeichnis

5.1 UML-ähnliche Sicht auf die wesentlichen Klassen, den gemeinsamen Service-Layer und ihre Beziehungen	11
10.1 Aktivitätsdiagramm des Simulationsablaufs für einen Akkutyp	24
10.2 Programmfluss von den Eingabedaten bis zu den Ausgaben	25
11.1 Höhenprofil der betrachteten Route	28
11.2 Zeitverlauf der Simulationsgrößen für den LiPo-Lauf	29
11.3 Vergleich des Ladezustands beider Akkutypen	29
11.4 Vergleich von Roh- und geglätteter Geschwindigkeit	30

Tabellenverzeichnis

7.1	Parameter der Geschwindigkeitsglättung (data/speed_smoothing_config.yaml) .	14
8.1	Parameter und Konstanten des Fahrzeugmodells (EBike, data/bike_config.yaml)	19
13.1	Abgedeckte Testbereiche	35
14.1	Ermittelte Kennzahlen des aktuellen Datensatzes	36
14.2	Vergleich ohne und mit Glättung	38

Zusammenfassung

Das Abschlussprojekt implementiert eine E-Bike-Simulation auf Basis realer GPS-Streckendaten. Aus einer CSV-Datei werden Zeit, Position, Höhe und Temperatur eingelesen und in Strecke, Geschwindigkeit, Beschleunigung, Steigung und Kurs umgerechnet. Daraus berechnet das Programm die nötige Antriebskraft, Leistung, das Raddrehmoment und den Motorstrom. Eine optionale Geschwindigkeitsglättung reduziert GPS-Rauschen und Beschleunigungsspitzen. Die Simulation führt ein LiPo- und ein NMC-Akkupack durch dieselbe Strecke und speichert Leistung, Drehmoment, Strom, Spannung und Ladezustand. Für den vorliegenden Datensatz ergeben sich 94.27 km Strecke, 4.55 h Fahrzeit und ein End-SoC von 0 % für beide Akkus, da der Akku während der Fahrt leer wird.

Kapitel 1

Einleitung

Das Abschlussprojekt untersucht, wie viel Energie ein E-Bike auf einer echten Route verbraucht. Grundlage ist keine künstliche Lastkurve, sondern eine reale GPS-Aufzeichnung mit Zeit, Position, Höhe und Temperatur. So fließt die tatsächliche Topografie und Fahrweise direkt in die Simulation ein.

Die benötigte Antriebskraft hängt von mehreren Größen ab: Geschwindigkeit, Beschleunigung, Steigung, Masse und Luftwiderstand. Weil GPS-Daten leicht rauschen, kann das zu unruhigen Geschwindigkeitswerten führen. Das Projekt bereitet die Messdaten deshalb auf, berechnet daraus die Fahrwiderstände und vergleicht damit zwei Akkutechnologien.

Der Fokus ist bewusst eng gehalten: Es wird kein vollständiges Fahrdynamikmodell mit Fahrerleistung, Wind, Antriebsregelung oder Motorwirkungsgrad simuliert. Stattdessen berechnet die Software aus den GPS-Daten die nötige mechanische Leistung und speist sie in ein LiPo- und ein NMC-Akkupack ein. Im Zentrum steht also nicht die Fahrzeugsimulation selbst, sondern die Frage, wie sich eine gegebene Fahrt auf unterschiedliche Akkus auswirkt.

1.1 Beitrag des Projekts

Das Programm verbindet Datenverarbeitung, Physik und Visualisierung in einer Pipeline. Die GPS-Rohdaten werden eingelesen, in Kinematikgrößen umgerechnet, optional geglättet und durch ein E-Bike- sowie zwei Akkumodelle verarbeitet. Die Ergebnisse erscheinen als HTML-Karten, PNG-Diagramme und Logdatei. Die Dokumentvorlage stammt aus **mcidoc_template**, der Code liegt im Repository des Projekts **abschlussprojekt_repo**.

1.2 Dokumentation im Bericht

Die folgenden Kapitel behandeln der Reihe nach: Anforderungen und Repository-Struktur, die objektorientierte Architektur, GPS- und Kinematikberechnung, Geschwindigkeitsglättung, das physikalische Modell, Akku- und Motormodelle, den Simulationsablauf und die Ausgaben. Zum Schluss folgen Tests, Ergebnisse und die Grenzen des Modells.

Kapitel 2

Funktionsweise des Programms im Überblick

Dieses Kapitel zeigt kurz, wie das Programm als Ganzes funktioniert. Die folgenden Kapitel gehen dann ins Detail.

2.1 Grundidee

Das Programm beantwortet eine Frage: *Wie viel Energie kostet eine echte Fahrt ein E-Bike, und wie unterscheiden sich dabei zwei Akkutypen?* Dazu geht es die GPS-Aufzeichnung Punkt für Punkt durch und berechnet daraus Kraft, Leistung, Drehmoment und Motorstrom. Dieser Strom wirkt dann auf ein Akkumodell, das Ladezustand und Spannung entsprechend fortschreibt.

Die Bewegung (Geschwindigkeit, Beschleunigung, Steigung) liegt bereits als GPS-Messung vor. Aus dieser bekannten Bewegung berechnet das Programm über das Freikörperdiagramm die dafür nötige Kraft und daraus den Strom.

2.2 Ablauf in neun Schritten

1. **CSV einlesen** (`gps_track.py`): Die Datei `data/final_project_input_data.csv` wird eingelesen. Jede Zeile enthält Zeit, Position, Höhe und Temperatur.
2. **Kinematik berechnen**: Aus je zwei Punkten entstehen Zeitdifferenz, Distanz (Haversine-Formel), Rohgeschwindigkeit, Steigung, Kurswinkel und Kompassrichtung.
3. **Geschwindigkeit optional glätten** (`speed_smoothing.py`): Kurze Zeitabstände lassen die Rohgeschwindigkeit stark schwanken. Ist die Glättung aktiviert, dämpft ein Medianfilter mit anschließender Exponentialglättung diese Spitzen. Die Beschleunigung wird danach aus der aktiven Geschwindigkeitskurve berechnet.
4. **Luftdichte schätzen** (`luftdichte.py`): Aus Höhe und Temperatur ergibt sich die aktuelle Luftdichte, statt pauschal mit einem Standardwert zu rechnen.

5. **Kraft, Leistung und Drehmoment berechnen** (`ebike.py`): Für jeden Punkt addiert `EBike.punkt_auswerten()` Luftwiderstand, Hangabtrieb, Beschleunigungskraft und Rollwiderstand. Daraus folgen Leistung und Drehmoment.
6. **Motorstrom bestimmen** (`motor.py`): Das Drehmoment wird über die Motorkonstante linear in einen Strom umgerechnet.
7. **Akku fortschreiben** (`battery_pack.py`, `lipo_battery.py`, `nmc_battery.py`, `akkutemperatur.py`): Der Strom wirkt über die Zeitspanne auf den Ladezustand. Die Temperatur korrigiert dabei Innenwiderstand und Kapazität, die Spannung ergibt sich aus der Leerlaufspannungskennlinie. Würde der Akku über 100 % geladen (z. B. bei starkem Bergabfahren), begrenzt `bremswiderstand.py` den Strom und wandelt den Rest in Wärme um.
8. **Simulation wiederholen** (`ebike_simulator.py`, `main.py`): Dieselbe Strecke läuft einmal mit LiPo- und einmal mit NMC-Akku, da beide von derselben Basisklasse erben.
9. **Ausgeben** (`plot_utils.py`, `gps_track.py`): Am Ende stehen Kennzahlen, HTML-Karten und PNG-Diagramme zur Verfügung.

2.3 Zusammenspiel der Bausteine

`main.py` ist der Einstiegspunkt: Es lädt Konfiguration und GPS-Daten und erzeugt die Objekte `GPSTrack`, `EBike`, `Motor` und zwei Akkus. `GPSTrack` liefert nur Bewegungsdaten, `EBike` nur die Fahrzeugphysik, `Motor` nur die Umrechnung von Drehmoment in Strom, die Akkuklassen nur ihr elektrisches Verhalten. `EBikeSimulator` verbindet diese vier Objekte, enthält aber selbst keine Physik – er steuert nur den Ablauf und sammelt die Ergebnisse. Diese klare Trennung ist die Grundlage der Architektur aus Kapitel 5.

2.4 Die zwei Konfigurationsdateien

Zwei YAML-Dateien steuern das Verhalten, ohne dass Code geändert werden muss: `data/bike_config.yaml` legt die Fahrzeugparameter fest (Tabelle 8.1), `data/speed_smoothing_config.yaml` die Glättung (Tabelle 7.1). Beide werden beim Start einmal geladen.

2.5 Ergebnis in einem Satz

Aus einer GPS-Aufzeichnung entsteht so ein nachvollziehbarer Weg von Position und Höhe über Kraft und Leistung bis zum Ladezustand zweier Akkus – die Zahlen dazu stehen in Kapitel 14.

Kapitel 3

Anforderungen

Das Projekt setzt die Kernfunktionen einer routenbasierten E-Bike-Simulation nach den Projektvorgaben um und ergänzt sie um mehrere Erweiterungen.

3.1 Implementierte Kernfunktionen

- Einlesen und Auswerten der GPS-Daten (`time`, `lat`, `lon`, `ele`, `temperature`).
- Berechnung von Geschwindigkeit, Beschleunigung, Leistung, Steigung, Drehmoment am Motor und Motorstrom bei bekannter Motorkonstante.
- Entwicklung des Ladezustands (SoC) über die gesamte Fahrt.
- Kenngrößen aus dem Gesamtdatensatz: Durchschnittsgeschwindigkeit, zurückgelegte Strecke, benötigte Zeit, Höhenmeter Anstieg/Abstieg und Maximalleistung.
- Simulation von zwei Akkutypen (LiPo und NMC) mit unterschiedlichen OCV-Kennlinien nach dem Ersatzschaltbild-Modell aus der Vorlesung.
- Grafische Darstellung von Geschwindigkeit, Leistung, Ladezustand und Höhenprofil über Zeit bzw. Strecke.
- Logging und Fehlerbehandlung, die verhindern, dass der Ladezustand über 100 % oder unter 0 % fällt (siehe Kapitel 9).
- Aktivitätsdiagramm des gesamten Simulationsablaufs (siehe Kapitel 10).
- `requirements.txt` mit allen benötigten Paketen sowie `README.md` mit Installations- und Ausführungsanleitung.

3.2 Umgesetzte Erweiterungen

Neben den Kernfunktionen implementiert das Projekt folgende Erweiterungen:

- Bestimmung der Luftdichte aus Höhe und Temperatur statt eines konstanten Werts (`luftdichte.py`).
- Simulation des Rollwiderstands als zusätzlicher Kraftanteil.
- Simulation der Akkutemperatur und ihres Einflusses auf Innenwiderstand und Kapazität (`akkutemperatur.py`).
- Simulation eines Bremswiderstands, der überschüssige Energie beim Laden dissipiert (`bremswiderstand.py`).
- Bestimmung der Kompassrichtung aus den GPS-Daten.
- Darstellung der Strecke und des Ladezustands auf interaktiven Karten mit `folium`.
- Reverse Geocoding: Umwandlung von GPS-Koordinaten in Adressen entlang der Strecke über Nominatim/OpenStreetMap (`GPSTrack.orte_ermitteln()`).
- Abfrage realer Wetterdaten (Temperatur, Wind, Niederschlag) für die Streckenpunkte über die Open-Meteo-API (`wetterdaten.py`).
- Unittest-Suite für Glättung, Kinematik und Simulator-Kompatibilität.
- Eine Web-Anwendung (Flask) zur Bedienung der Simulation im Browser, inklusive automatischer Ein-Faktor-Parameterstudien für Fahrzeugparameter wie Masse, Luftwiderstandsfläche oder Raddurchmesser (`webapp/`).
- Automatisierte Erstellung dieses Berichts als $\text{\LaTeX}$ -Dokument.

3.3 Technische Randbedingungen

Die Software läuft lokal mit Python in einer virtuellen Umgebung. Benötigt werden die Pakete aus `requirements.txt` sowie LuaLaTeX für den Bericht. Ausgaben landen unter `output/`; für den Bericht liegen Kopien der wichtigsten Plots zusätzlich unter `report/figures/generated/`.

Kapitel 4

Projektstruktur

Das Repository ist klar aufgeteilt: Die fachliche Logik liegt in einzelnen Python-Modulen, die Web-Anwendung in `webapp/` und die Dokumentation in `report/`. So bleiben GPS-Verarbeitung, Fahrzeugphysik, Motorlogik, Akkuverhalten, Web-Oberfläche und Ausgaben voneinander getrennt.

4.1 Übersicht der Komponenten

Die wichtigsten Bausteine des Projekts sind:

- `main.py`: Einstiegspunkt für die Kommandozeile; lädt Konfigurationen und startet die Simulation.
- `app.py`: Startet die Flask-Anwendung über `create_app()`.
- `gps_track.py`: Liest GPS-Daten ein und berechnet Distanz, Geschwindigkeit, Steigung, Kurs und Karten.
- `speed_smoothing.py`: Lädt die Glättungskonfiguration und glättet die Geschwindigkeitskurve zeitbasiert.
- `ebike.py`: Enthält die Fahrphysik mit Luft-, Hang-, Roll- und Beschleunigungsanteilen.
- `motor.py`: Rechnet Drehmoment in Motorstrom um.
- `ebike_simulator.py`: Führt die Fahrt Schritt für Schritt aus und schreibt Ergebnisprofile.
- `battery_base.py` und `battery_pack.py`: Abstrakte Akku-Schnittstelle und gemeinsame Implementierung.
- `lipo_battery.py` und `nmc_battery.py`: Akkutypen mit spezifischer OCV-Kennlinie und Parametern.
- `akkutemperatur.py`: Temperaturkorrektur von Innenwiderstand und Kapazität.
- `luftdichte.py`: Schätzt die Luftdichte aus Höhe und Temperatur.

- `bremswiderstand.py`: Berechnet die dissipierte Leistung bei Rekuperationsbegrenzung.
- `plot_utils.py`: Erzeugt Diagramme für Verlauf und Vergleich.
- `wetterdaten.py`: Ruft Wetterdaten über die Open-Meteo-API ab.
- `webapp/`: Flask-Blueprint, Services, Templates und Web-Ausgaben.
- `tests/`: Automatisierte Tests für Glättung, GPS-Verarbeitung und Web-Anbindung.
- `data/`: Eingabedaten und Konfigurationsdateien.
- `output/`: Laufzeit-Ausgaben des Programms.
- `report/`: LaTeX-Bericht und erzeugte PDF-Dokumentation.

4.2 Zentrale Klassen und Funktionen

Die zentralen Klassen und Funktionen lassen sich wie folgt zusammenfassen:

- **GPSTrack**: Liest die CSV ein und berechnet sofort die gesamte Kinematik. Wichtige Methoden sind `gesamstrecke_km()`, `durchschnittsgeschwindigkeit_kmh()`, `gesamtzeit_s()`, `hoehenmeter_anstieg()`, `hoehenmeter_abstieg()`, `kennzahlen_ausgeben()`, `smoothing_kennzahlen()`, `karte_erstellen(...)` und `orte_ermitteln(anzahl_wegpunkte=8)`.
- **SpeedSmoothingConfig**: Lädt die Glättungskonfiguration über `from_yaml(path)` und stellt die umgerechnete Plausibilitätsgrenze `max_reasonable_speed_ms` bereit.
- **EBike**: Erzeugt ein Fahrzeugobjekt aus `data/bike_config.yaml`. Dazu gehören `antriebskraft_N(v, a, phi, rho)`, `leistung_W(F, v)` und `drehmoment_Nm(F)`.
- **Motor**: Speichert die Motorkonstante k_m und rechnet mit `get_current_draw(torque_Nm)` Drehmoment in Strom um.
- **BatteryBase**: Definiert die abstrakten Methoden `apply_current(...)` und `voltage(...)` für Akku-Modelle.
- **BatteryPack**: Enthält die gemeinsame Akku-Logik mit `set_temperatur(temperature_c)`, `apply_current(current, duration)`, `voltage(current)`, `is_empty()` und `is_full()`.
- **LiPoBatteryPack** und **NMCBatteryPack**: Spezialisieren **BatteryPack** mit eigener OCV-Kennlinie über `_ocv(soc)` und individuellen Packparametern.
- **EBikeSimulator**: Verknüpft **Track**, **Fahrzeug**, **Motor** und **Akku**. Die Methode `simulate()` führt die Strecke punktweise aus; weitere Auswertungen liefern `maximalleistung_W()`, `bremswiderstand_energie_Wh`, `endladezustand_prozent()` und `zusammenfassung_ausgeben()`.

4.3 Dateien und Ausgaben

Die Eingabedaten liegen unter `data/`. Ergebnisse entstehen zur Laufzeit unter `output/`; die Web-Anwendung nutzt eigene Unterverzeichnisse unter `output/web/`. Für den Bericht werden die PNGs zusätzlich nach `report/figures/generated/` kopiert, damit der PDF-Build unabhängig vom letzten Programmlauf funktioniert.

Kapitel 5

Softwarearchitektur und OOP

Die Architektur trennt Datenquelle, physikalisches Modell, Energieversorgung, Visualisierung und Web-Oberfläche klar voneinander. Die Klassen arbeiten über Assoziationen und Vererbung zusammen, jede bleibt aber für ihren eigenen Bereich zuständig.

5.1 Objektorientierte Struktur

`BatteryBase` definiert die abstrakte Schnittstelle für alle Akku-Modelle. `BatteryPack` liefert die gemeinsame Umsetzung mit SoC-Integration und Temperaturkorrektur. `LiPoBatteryPack` und `NMCBatteryPack` ändern nur die Leerlaufspannungskennlinie und die Packparameter. So kann `EBikeSimulator` jeden Akkutyp gleich behandeln, ohne ihn genau zu kennen.

`GPSTrack` liefert die Daten, `EBike` die Fahrphysik, `Motor` die Umrechnung von Drehmoment in Strom. Der Simulator verbindet diese Objekte und führt die Strecke Schritt für Schritt aus.

5.2 Klassendiagramm

Das Diagramm zeigt die Struktur aus dem Code: Beide Einstiegspunkte (`main.py` und `webapp/routes.py`) rufen denselben Service-Layer `run_simulation()` auf, der `EBikeSimulator` mit `GPSTrack`, `EBike`, `Motor` und dem gewählten Akku verbindet. `GPSTrack` nutzt zusätzlich `SpeedSmoothingConfig` für die optionale Glättung. Bei den Batterien zeigt sich die Vererbung nach unten. Die Pfeile sind bewusst einfach gehalten, da der Fokus auf der Architektur liegt, nicht auf vollständiger UML-Notation.

5.3 Daten- und Kontrolltrennung

Datenverarbeitung und Simulation sind entkoppelt. Das Programm liest die CSV, bereitet sie in `GPSTrack` auf, erzeugt E-Bike und Motor und simuliert dann jeden Akkutyp separat. Diagramme und Karten sind reine Ausgaben und beeinflussen das Modell nicht.

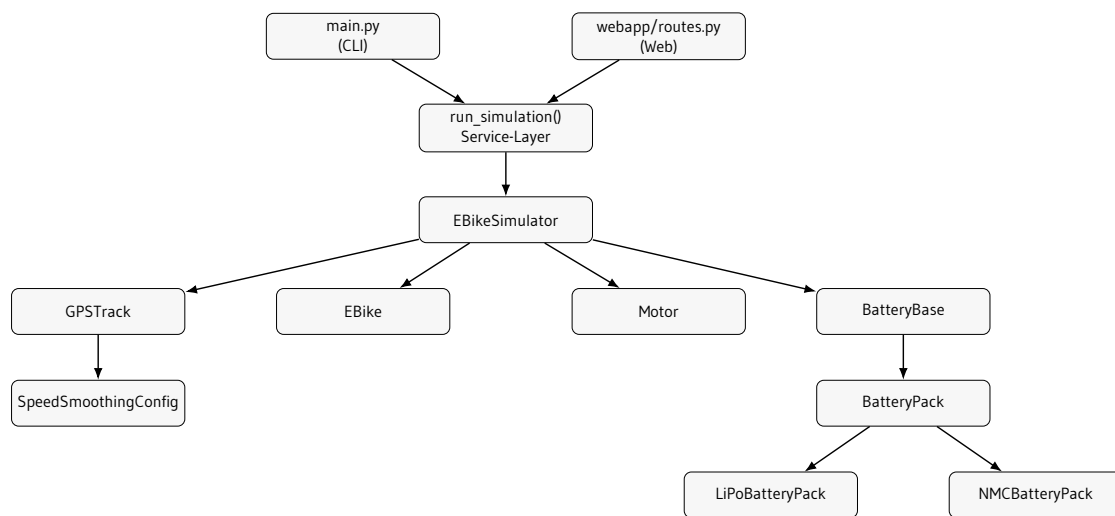


Abbildung 5.1: UML-ähnliche Sicht auf die wesentlichen Klassen, den gemeinsamen Service-Layer und ihre Beziehungen

5.4 Ein gemeinsamer Service-Layer für CLI und Web-App

Sowohl `main.py` als auch die Flask-Anwendung nutzen denselben Simulationsservice in `webapp/services/simulation.py`. Dort wird die Strecke geladen, das E-Bike-Modell aufgebaut, der Simulator für die gewählten Akkus ausgeführt und das Ergebnis zusammengefasst. Die Web-Routen in `webapp/routes.py` übernehmen nur Formularverarbeitung, Uploads, Ergebnisanzeige und den Aufruf von `run_simulation()` beziehungsweise `run_parameter_study()`; sie enthalten selbst keine zweite Simulation.

`app.py` startet die Flask-Anwendung über `create_app()` aus `webapp/__init__.py`. Die Blueprint-Routen erzeugen pro Lauf ein eigenes Verzeichnis unter `output/web/` und lesen die dort erzeugten CSV-, JSON- und Karten-Dateien wieder ein.

Kapitel 6

GPS- und Kinematikberechnung

Die GPS-Eingabe ist eine semikolonseparierte CSV mit den Spalten `time`, `lat`, `lon`, `ele` und `temperature`. Die Zeitstempel liest `pandas.to_datetime` als UTC ein **pandas_docs**. Die Zeilen müssen bereits zeitlich sortiert sein, die Berechnung nutzt dann nur aufeinanderfolgende Punkte.

6.1 Zeit und Distanz

Für zwei aufeinanderfolgende Punkte $i - 1$ und i gilt im Code:

$$\Delta t_i = t_i - t_{i-1} \quad (6.1)$$

$$\Delta s_i = 2R \arcsin(\sqrt{a}) \quad (6.2)$$

mit

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_{i-1}) \cos(\varphi_i) \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (6.3)$$

und $R = 6\,371\,000$ m als mittlerem Erdradius. Diese Haversine-Formel ist für die Distanzberechnung vorgegeben und wird direkt in `GPSTrack_distanz_haversine()` umgesetzt.

Aus Distanz und Zeit folgt die Rohgeschwindigkeit:

$$v_i = \frac{\Delta s_i}{\Delta t_i} \quad (6.4)$$

Ist $\Delta t_i \leq 0$, setzt das Projekt die Rohgeschwindigkeit auf 0 m/s und markiert das Intervall als ungültig. So vermeidet die Berechnung eine Division durch null, ohne Datenpunkte zu verlieren.

6.2 Steigung, Kurs und Kompassrichtung

Die Höhendifferenz wird als

$$\Delta h_i = h_i - h_{i-1} \quad (6.5)$$

verwendet. Daraus entsteht die Steigung in Grad als

$$\varphi_i = \arctan\left(\frac{\Delta h_i}{\Delta s_i}\right) \quad (6.6)$$

falls $\Delta s_i > 0$ ist, sonst 0. Der Kurswinkel folgt aus Breiten- und Längengrad über die Anfangskurs-Formel und wird auf $[0, 360)$ normalisiert. Daraus ergibt sich die Kompassrichtung: N, NO, O, SO, S, SW, W oder NW.

6.3 Beschleunigung

Die Beschleunigung entsteht nicht aus einer einfachen Differenz zweier aufeinanderfolgender Geschwindigkeitswerte, sondern segmentweise aus der aktiven Geschwindigkeitskurve. Der Grund: Bei ungleichmäßigen Zeitabständen – wie sie bei GPS-Aufzeichnungen üblich sind – würde eine reine Vorwärtsdifferenz das Ergebnis unnötig empfindlich gegenüber genau dem Punkt machen, an dem sie ausgewertet wird. `beschleunigung_aus_geschwindigkeit()` verwendet deshalb eine zentrale Differenz: Für einen Punkt i wird die Geschwindigkeitsänderung zwischen dem Vorgänger- und dem Nachfolgerpunkt durch die Zeit zwischen den beiden Intervallmitten geteilt. Das mittelt den Einfluss beider Nachbarn und reagiert dadurch weniger empfindlich auf einen einzelnen verrauschten Messwert als eine reine Vorwärts- oder Rückwärtsdifferenz.

An den Segmentgrenzen gibt es keinen Vorgänger bzw. Nachfolger im selben Segment; dort setzt die Funktion die Beschleunigung am ersten Punkt auf null und verwendet am letzten Punkt eine einseitige (rückwärtige) Differenz. Segmente werden dabei nicht überschritten: Eine Beschleunigung wird nie über eine grosse Zeitlücke hinweg berechnet, weil das einen physikalisch bedeutungslosen Wert ergeben würde (z. B. eine sehr kleine Beschleunigung über eine mehrminütige Aufzeichnungspause).

6.4 Wichtige DataFrame-Spalten

Diese Spalten bilden die Grundlage für die weiteren Kapitel: `dt_s`, `distanz_m`, `steigung_grad`, `kurs_grad`, `kompassrichtung`, `geschwindigkeit_roh_ms`, `geschwindigkeit_geglaettet_ms`, `geschwindigkeit_ms`, `beschleunigung_roh_ms2`, `beschleunigung_geglaettet_ms2`, `beschleunigung_ms2` sowie die Filterspalten der Glättung. Der Simulator nutzt später nur `geschwindigkeit_ms` und `beschleunigung_ms2`.

Kapitel 7

Geschwindigkeitsglättung

GPS-Positionen schwanken oft leicht, besonders bei kurzen Zeitintervallen. Da die Geschwindigkeit hier aus Distanz durch Zeit berechnet wird, erzeugen kleine Zeitdifferenzen schnell hohe Spitzen – die wiederum auf Beschleunigung und Leistung durchschlagen. Das Projekt bietet daher eine zeitbasierte Glättung, die sich in `data/speed_smoothing_config.yaml` ein- oder ausschalten lässt.

7.1 Konfigurationsparameter

Die aktuelle Konfiguration enthält diese Werte:

Tabelle 7.1: Parameter der Geschwindigkeitsglättung (`data/speed_smoothing_config.yaml`)

Parameter	Aktueller Wert	Bedeutung
<code>enabled</code>	<code>true</code>	Schaltet die Glättung ein oder aus; bei <code>false</code> bleiben die Rohwerte aktiv.
<code>min_interval_s</code>	0.5 s	Untere Schranke für gültige Filterstützstellen; kürzere Intervalle gelten als zu unsicher für eine Geschwindigkeitsberechnung und werden ausgeschlossen.
<code>max_gap_s</code>	30.0 s	Zeitlücke, ab der ein neues Segment beginnt, damit über Aufzeichnungslücken hinweg nicht geglättet oder interpoliert wird.
<code>median_window_s</code>	15.0 s	Zeitfenster des zeitbasierten Medianfilters; größer bedeutet stärkere Glättung, aber trägere Reaktion auf echte Geschwindigkeitsänderungen.
<code>time_constant_s</code>	3.0 s	Zeitkonstante τ der exponentiellen Glättung; steuert, wie schnell die geglättete Kurve der Rohkurve folgt.
<code>max_reasonable_speed_kmh</code>	60.0 km/h	Plausibilitätsgrenze für Stützstellen; darüber liegende Rohwerte gelten als Ausreißer und werden aus der Filterbasis entfernt.

7.2 Ablauf der Glättung

Der Algorithmus arbeitet segmentweise und führt die folgenden Schritte aus:

1. Rohgeschwindigkeit berechnen.
2. Ungültige Intervalle erkennen, also $\Delta t \leq 0$.
3. Sehr kurze Intervalle als ungeeignete Stützstellen markieren.
4. Grosse Zeitlücken in getrennte Segmente aufteilen.
5. Unplausible Geschwindigkeiten als Ausreisser markieren.
6. Einen zeitbasierten Medianfilter anwenden.
7. Fehlende Werte nur innerhalb des Segments interpolieren.
8. Eine zeitabhängige exponentielle Glättung vorwärts ausführen.
9. Die Glättung rückwärts wiederholen.
10. Beide Verläufe mitteln.
11. Die Beschleunigung aus der aktiven Geschwindigkeitskurve berechnen.

Warum Segmente statt einer durchgehenden Kurve? Eine grosse Zeitlücke (länger als `max_gap_s`) bedeutet meist eine echte Aufzeichnungspause, z. B. ein Stopp ohne GPS-Fix. Würde der Filter darüber hinweg glätten oder interpolieren, würde er eine Bewegung erfinden, die nie stattgefunden hat. Deshalb beginnt an solchen Lücken ein neues Segment, und jedes Segment wird unabhängig gefiltert.

Warum zuerst Median, dann Exponentialfilter? Die beiden Filter lösen unterschiedliche Probleme. Der zeitbasierte Median entfernt einzelne, kurze Ausreisser robust, weil ein Median durch einen einzelnen extremen Wert kaum verschoben wird – anders als ein Mittelwert. Erst danach glättet die exponentielle Filterung die verbleibende, kleinräumige Unruhe der Kurve. Würde man nur exponentiell glätten, würden einzelne GPS-Spitzen die Kurve noch mehrere Sekunden lang nachziehen.

Warum vorwärts UND rückwärts filtern? Eine einfache exponentielle Glättung reagiert erst NACH einer Geschwindigkeitsänderung – sie hinkt der echten Kurve zeitlich hinterher (Phasenverzögerung). Filtert man dieselbe Formel zusätzlich auf der umgekehrten Zeitachse, "hinkt" dieser zweite Verlauf in die andere Richtung, er reagiert quasi zu früh. Der Mittelwert aus beiden Richtungen hebt diese Verzögerung weitgehend auf, sodass die geglättete Kurve zeitlich zur echten Bewegung passt statt ihr nur verzögert zu folgen.

Was passiert, wenn ein Segment keine gültige Stützstelle hat? Bleiben in einem kurzen Segment z. B. nur Ausreisser oder zu kurze Intervalle übrig, kann nicht sinnvoll gefiltert werden. In diesem Fall verwendet das Programm für dieses Segment die Rohgeschwindigkeit unverändert weiter und loggt eine Warnung, statt mit leeren oder erfundenen Werten weiterzurechnen.

7.3 Mathematische Beschreibung

Die zeitabhängige Glättung verwendet den Faktor

$$\alpha_i = 1 - \exp\left(-\frac{\Delta t_i}{\tau}\right) \quad (7.1)$$

mit der Zeitkonstante τ . Daraus folgt die Rekursion

$$v_{\text{glatt},i} = v_{\text{glatt},i-1} + \alpha_i (v_i - v_{\text{glatt},i-1}) \quad (7.2)$$

Die Rückwärtsfilterung nutzt dieselbe Formel auf der umgekehrten Zeitachse; anschliessend werden beide Verläufe gemittelt.

Die Konfiguration wird aus YAML geladen **pyyaml_docs** und vor der Nutzung auf Plausibilität geprüft.

7.4 Aktive und passive Spalten

Das Projekt unterscheidet zwischen Roh-, geglätteter und aktiver Größe. `geschwindigkeit_roh_ms` bleibt immer erhalten, `geschwindigkeit_geglaettet_ms` zeigt das Filterergebnis, `geschwindigkeit_ms` ist die für die Simulation aktive Kurve. Analog gilt das für `beschleunigung_roh_ms2`, `beschleunigung_geglaettet_ms2` und `beschleunigung_ms2`. Es gibt keine pauschale Begrenzung oder Kappung – nur Filterstützstellen und Segmente werden gezielt behandelt.

Kapitel 8

Physikalisches E-Bike-Modell

Das Fahrzeugmodell in `ebike.py` berechnet für jeden GPS-Punkt die nötige Kraft aus Fahrwiderständen, Beschleunigung und Steigung, wie in der Aufgabenstellung über das Freikörperdiagramm vorgegeben. Geschwindigkeit, Beschleunigung und Steigung liegen dafür bereits aus der GPS-Auswertung vor (Kapitel 6, 7).

8.1 Einzelkräfte

Der Luftwiderstand folgt der üblichen quadratischen Drag-Gleichung **nasa_drag**:

$$F_{\text{Luft}} = \frac{1}{2} \rho c_w A v^2 \quad (8.1)$$

Dabei ist ρ die Luftdichte, $c_w A$ die aerodynamische Kennzahl und v die Geschwindigkeit. Für ρ nutzt das Projekt entweder den aktuellen Wert aus `luftdichte.py` oder eine Standarddichte, falls Höhe und Temperatur fehlen. `luftdichte.py` kombiniert dafür zwei physikalische Zusammenhänge: die barometrische Höhenformel, die beschreibt, wie der Luftdruck mit der Höhe abnimmt,

$$p(h) = p_0 \exp\left(-\frac{gh}{R_s T}\right), \quad (8.2)$$

und das ideale Gasgesetz, das aus Druck und Temperatur die Dichte liefert,

$$\rho = \frac{p(h)}{R_s T}. \quad (8.3)$$

Dabei ist p_0 der Luftdruck auf Meereshöhe, R_s die spezifische Gaskonstante trockener Luft, T die Temperatur in Kelvin und h die Höhe. So sinkt die Luftwiderstandskraft realistisch bei größerer Höhe (dünnere Luft), statt mit einer festen Standarddichte zu rechnen.

Die Hangabtriebskraft lautet

$$F_{\text{Hang}} = mg \sin(\varphi) \quad (8.4)$$

und die Beschleunigungskraft

$$F_{\text{Beschleunigung}} = ma \quad (8.5)$$

mit der Gesamtmasse m , der Erdbeschleunigung g , der Steigung φ und der Beschleunigung a .
Der Rollwiderstand wird im Code als

$$F_{\text{Roll}} = c_{\text{R}} m g \cos(\varphi) \quad (8.6)$$

modelliert. Die Masse ist die Summe aus Fahrer- und Fahrradmasse.

8.2 Gesamtkraft und Leistung

`EBike.antriebskraft_N()` bildet die Summe

$$F_{\text{Antrieb}} = F_{\text{Luft}} + F_{\text{Hang}} + F_{\text{Beschleunigung}} + F_{\text{Roll}} \quad (8.7)$$

– genau die im Code verwendete Kraft. Daraus folgen mechanische Leistung

$$P = Fv \quad (8.8)$$

und Raddrehmoment

$$M = Fr \quad (8.9)$$

mit dem Radradius r .

8.3 Parameter

Die Parameter stammen aus `data/bike_config.yaml`: Fahrer- und Radmasse zusammen 80 kg, aerodynamische Kennzahl $0,5625 \text{ m}^2$, Raddurchmesser 27 cm, Rollwiderstandskoeffizient 0,005. Zusammen mit der GPS-Kinematik ergibt sich daraus die Leistungsanforderung entlang der Strecke.

Tabelle 8.1: Parameter und Konstanten des Fahrzeugmodells (EBike, data/bike_config.yaml)

Parameter/Konstante	Wert	Kommentar
masse_fahrer_kg	70.0 kg	Fahrgewicht; geht zusammen mit masse_rad_kg in masse_gesamt_kg ein, das in allen vier Kraftanteilen verwendet wird.
masse_rad_kg	10.0 kg	Fahrzeuggewicht; wird zur Fahrer Masse addiert, da im Modell nur die Gesamtmasse relevant ist.
cw_a_m2	0.5625 m ²	Produkt aus Luftwiderstandsbeiwert c_w und Stirnfläche A ; skaliert die Luftwiderstandskraft quadratisch mit der Geschwindigkeit.
raddurchmesser_inch	27.0 Zoll	Wird im Konstruktor in den Radradius radradius_m umgerechnet (Zoll $\rightarrow$ Meter, Durchmesser $\rightarrow$ Radius) und bestimmt so das Drehmoment $M = Fr$.
rollwiderstandkoeffizient	0.005	Koeffizient c_R des Rollwiderstands; skaliert linear mit der Normalkraft $mg \cos(\varphi)$.
RHO_LUFT (Klassenkonstante)	1.225 kg/m ³	Standard-Luftdichte auf Meereshöhe; wird verwendet, wenn keine höhen-/temperaturabhängige Dichte aus luftdichte.py verfügbar ist.
G (Klassenkonstante)	9.81 m/s ²	Erdbeschleunigung; geht in Hangabtriebs- und Rollwiderstandskraft ein.

Kapitel 9

Motor- und Akkumodelle

9.1 Motor

Die Klasse `Motor` bildet die geforderte lineare Beziehung zwischen Raddrehmoment und Motorstrom ab:

$$I = \frac{M}{k_m} \quad (9.1)$$

Die Motorkonstante ist in N m/A angegeben. Der Motor ist als eigene Klasse modelliert, damit sich diese Beziehung später leicht austauschen lässt.

9.2 Abstrakte Akku-Schnittstelle

`BatteryBase` definiert nur das Interface: Jede Akku-Klasse muss Ladezustand und Klemmenspannung bereitstellen. Der Simulator arbeitet dadurch auf dem Basistyp, nicht auf einer konkreten Unterklasse.

9.3 Gemeinsame Akku-Funktionalität

`BatteryPack` beschreibt ein Ersatzschaltbild aus Leerlaufspannung und Innenwiderstand. Der Ladezustand wird per Stromintegration aktualisiert:

$$\text{SoC}_{k+1} = \text{SoC}_k - \frac{I \Delta t}{C_{\text{nom}}} \quad (9.2)$$

Positiver Strom entlädt den Akku, negativer lädt ihn. Die Klemmenspannung ergibt sich als

$$U = U_{\text{OC}}(\text{SoC}) - R_{\text{int}} I \quad (9.3)$$

und wird temperaturkorrigiert, sobald eine Temperatur gesetzt ist.

Kältere Temperaturen erhöhen den Innenwiderstand und senken die nutzbare Kapazität relativ

zu 25 °C. Beide Korrekturen sind nach unten begrenzt, damit extreme Temperaturen nicht zu unrealistischen Werten führen.

Warum Logging und eine Exception statt stiller Begrenzung? Ein SoC über 100 % oder unter 0 % ist physikalisch nicht möglich. `apply_current()` fängt beide Fälle aktiv ab: Wird ein leerer Akku weiter entladen, wirft die Methode einen Fehler. Wird der Ladezustand über 100 % gedrückt, leitet der Simulator den Überschuss über den Bremswiderstand ab und protokolliert dies.

9.4 LiPo und NMC

LiPo- und NMC-Akkus erben von `BatteryPack` und unterscheiden sich nur in Kennlinie und Innenwiderstand. Beide werten tabellarische Datenblattwerte mit linearer Interpolation aus **numpy_docs**, sodass sich ihr Verhalten unter identischer Last direkt vergleichen lässt.

Die OCV-Kennlinien unterscheiden sich in den Stützstellen, werden aber jeweils mit `numpy.interp` ausgewertet. Dadurch ergibt sich unter gleicher Last für LiPo meist eine etwas höhere Leerlaufspannung als für NMC.

9.5 Wichtige Attribute und Parameter

Die zentralen Kennwerte von Motor- und Akkumodell sind:

- `Motor.km`: Motorkonstante, die Drehmoment und Strom verknüpft.
- `BatteryPack.C_nom`: Nennkapazität des Packs; intern von Ah in As umgerechnet.
- `BatteryPack.R_int`: Innenwiderstand des Packs pro Zelle.
- `BatteryPack.soc`: Ladezustand im Bereich von 0.0 bis 1.0.
- `BatteryPack.Vmin` und `BatteryPack.Vmax`: Spannungsgrenzen bei leerem beziehungsweise vollem Akku.
- `BatteryPack.n_parallel`: Anzahl paralleler Zellstränge.
- `letzte_dissipierte_leistung_W`: Zuletzt über den Bremswiderstand abgeführte Leistung.
- `REFERENZ_TEMPERATUR_C`: Referenztemperatur für alle Korrekturen.
- `WIDERSTAND_TEMP_KOEFFIZIENT_PRO_GRAD`: Widerstandszunahme je Grad unterhalb der Referenz.
- `KAPAZITAET_TEMP_KOEFFIZIENT_PRO_GRAD`: Kapazitätsabnahme je Grad unterhalb der Referenz.

- MIN_WIDERSTAND_FAKTOR und MIN_KAPAZITAET_FAKTOR: Untere Grenze der Temperaturkorrekturen.

9.6 Bremswiderstand und Rekuperation

Entsteht beim Bergabfahren oder Bremsen ein negativer Motorstrom, steigt der Ladezustand. Würde er über 100 % steigen, begrenzt das Programm den Strom und wandelt den Rest über den Bremswiderstand in Wärme um. Im aktuellen Datensatz tritt dieser Fall nicht auf; der Wert bleibt in beiden Läufen bei null.

Kapitel 10

Simulationsablauf

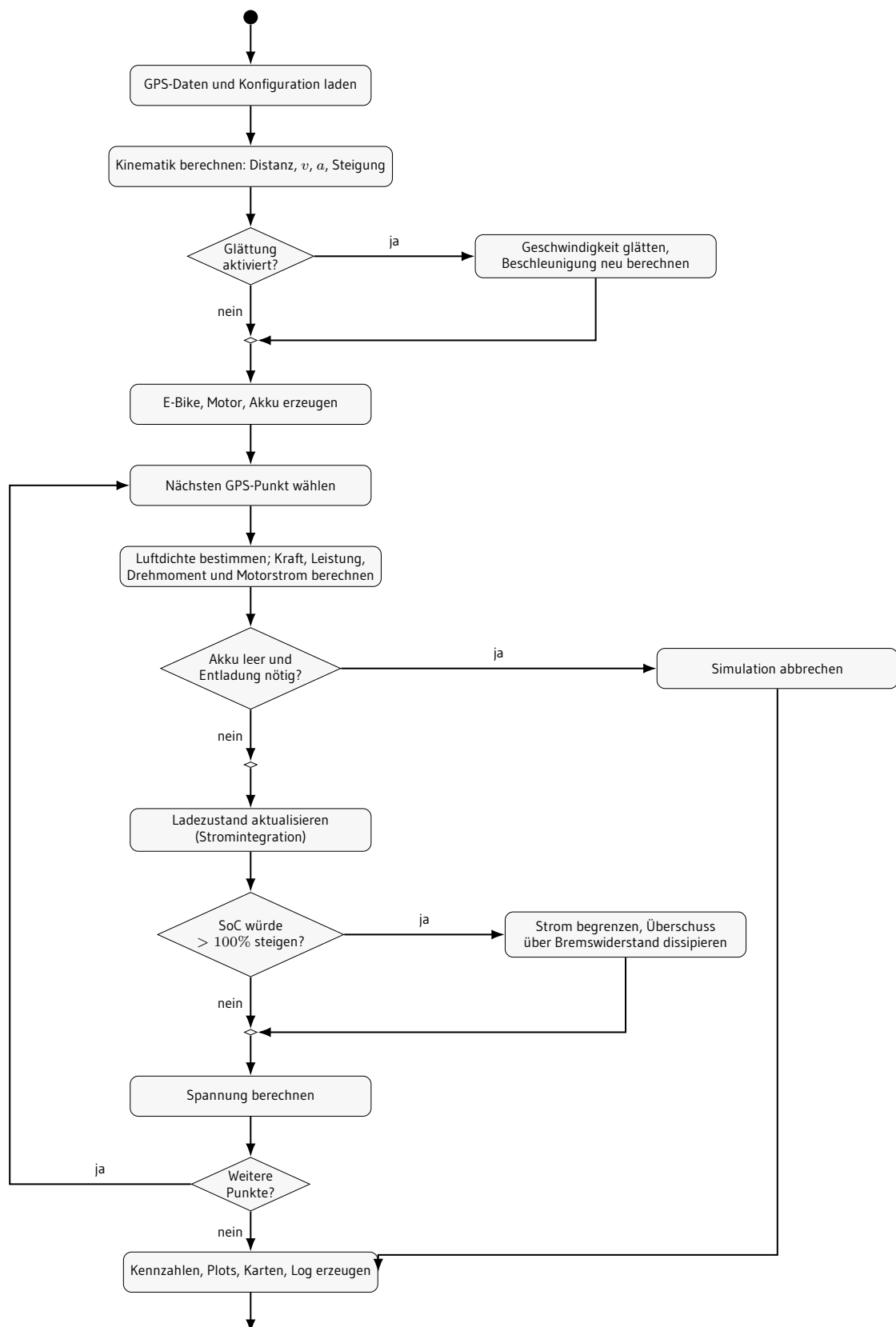
10.1 Aktivitätsdiagramm der Simulation

Abbildung 10.1 zeigt den vollständigen Ablauf einer Simulation für einen Akkutyp, von den Eingabedaten bis zu den Ausgaben. `BatteryPack.apply_current()` prüft dabei zuerst, ob der Akku leer ist und eine Entladung versucht wird (Abbruch), und erst danach, ob die Ladung 100 % überschreiten würde (Bremswiderstandslogik) – genau in dieser Reihenfolge bildet das Diagramm es ab.

10.2 Warum bricht die Simulation bei leerem Akku ab, statt weiterzurechnen?

`EBikeSimulator.simulate()` verarbeitet die Strecke Punkt für Punkt und hält dabei den zuletzt gültigen Index fest. Wirft der Akku beim Entladeversuch einen Fehler (siehe Kapitel 9), stoppt die Schleife an genau dieser Stelle: Das zurückgegebene Ergebnis-DataFrame enthält dann nur die Punkte bis zum Abbruch, nicht die gesamte Strecke. Diese Entscheidung spiegelt die Realität wider – ein leerer Akku kann keine weitere Fahrt antreiben –, und sie ist der Grund, warum in Kapitel 14 beide Akkus vor dem Streckenende auf 0 % SoC fallen: Die Simulation bricht dort ab, wo der Akku tatsächlich leer wäre, statt unrealistisch mit negativem Ladezustand weiterzulaufen.

Die Luftdichte wird pro Punkt einzeln bestimmt und nicht einmalig für die ganze Strecke: Höhe und Temperatur ändern sich während der Fahrt, und schlägt die Berechnung für einen einzelnen Punkt fehl (z. B. wegen fehlender Werte), fängt der Simulator das ab und verwendet für diesen einen Punkt die Standarddichte, statt die gesamte Simulation abzubrechen.

**Abbildung 10.1:** Aktivitätsdiagramm des Simulationsablaufs für einen Akkutyp

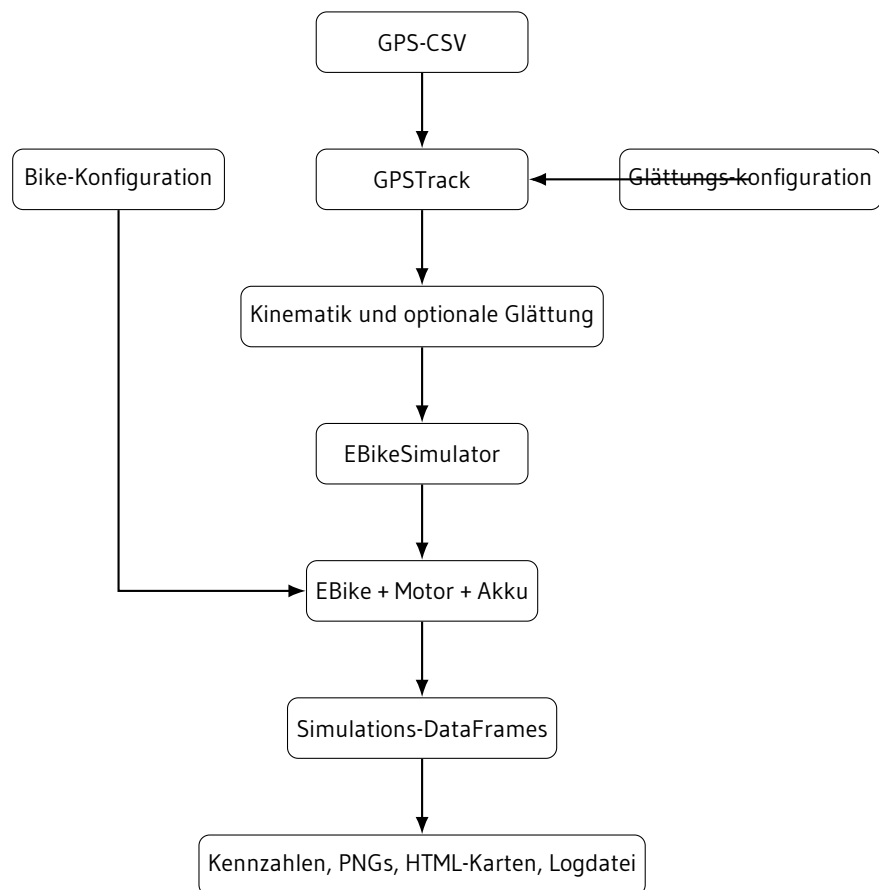


Abbildung 10.2: Programmfluss von den Eingabedaten bis zu den Ausgaben

10.3 Datenfluss

10.4 Ablauf in `main.py`

Der Programmstart legt die Ausgabeverzeichnisse an und startet das Logging. Danach lädt das Programm Glättungskonfiguration und GPS-Track, gibt die Streckenkennzahlen aus und schreibt eine Karte mit der Streckenfarbe nach Geschwindigkeit.

Als Nächstes entstehen E-Bike, Motor und zwei Akkupacks. Der Simulator läuft dann zweimal – einmal für LiPo, einmal für NMC – auf derselben Strecke. Nach jedem Lauf speichert das Programm eine SoC-Karte und das Ergebnis-DataFrame. Zum Schluss erstellt `plot_utils.py` die Diagramme.

10.5 Übergebene Daten

Zwischen den Komponenten wandern nur DataFrames, einfache Zahlen und Klassenobjekte. `GPSTrack` liefert die aufbereiteten Spalten, `EBikeSimulator` ergänzt Leistungs- und Akkuwerte. So bleibt die Schnittstelle einfach und testbar.

Kapitel 11

Visualisierung und Ausgaben

Das Projekt erzeugt zwei Ausgabearten: interaktive HTML-Karten mit Folium und statische PNG-Diagramme mit Matplotlib. Beide greifen nur auf bereits berechnete Daten zu.

11.1 HTML-Karten: wie der Streckenplot funktioniert

Die Karten entstehen in `GPSTrack.karte_erstellen()` mit folium **folium_docs**, das eine interaktive Leaflet-Karte im Browser erzeugt. Die Strecke wird dabei nicht als eine einzelne Linie gezeichnet, sondern in einzelne Segmente zwischen je zwei aufeinanderfolgenden GPS-Punkten zerlegt. Jedes Segment bekommt über eine lineare Farbskala (`branca.colormap`, blau-grün-gelb-rot) eine eigene Farbe, je nach Wert einer wählbaren Spalte – standardmäßig die Geschwindigkeit, bei den SoC-Karten stattdessen der Ladezustand. So zeigt die Streckenfarbe direkt, wo z. B. schnell gefahren wurde oder wo der Akku bereits weit entladen war.

Start- und Zielpunkt erhalten eigene Marker (grün/rot). Wurde zuvor `orte_ermitteln()` aufgerufen, zeigen deren Popups die per Reverse Geocoding ermittelte Adresse statt nur „Start“/„Ziel“, und zusätzliche blaue Marker markieren die abgefragten Zwischenpunkte. Als Kartenhintergrund dient standardmäßig `cartodbpositron` statt des klassischen OpenStreetMap-Stils: Der offizielle OSM-Kachelserver verlangt einen Referer-Header, den Browser beim direkten Öffnen einer lokalen HTML-Datei nicht mitschicken – die Kacheln würden sonst mit einem Zugriffsfehler abgelehnt.

Die Strecke liegt so als farbcodierte Linie in `output/karte_strecke.html`. Der Simulator erzeugt zusätzlich zwei SoC-Karten, `output/karte_soc_lipo.html` und `output/karte_soc_nmc.html`, indem dieselbe Funktion auf das jeweilige Ergebnis-DataFrame mit der Spalte `soc` angewendet wird. Alle Karten lassen sich direkt im Browser öffnen.

11.2 PNG-Diagramme

Die Plots liegen unter `output/plot/`, erzeugt mit matplotlib **matplotlib_docs**. `plot_utils.py` baut dafür die x-Achse aus der kumulierten Distanz (`distanz_m.cumsum()`) bzw. aus der seit Fahrt-

beginn vergangenen Zeit; die y-Werte kommen direkt aus den in Kapitel 8 und 9 berechneten Spalten. Für den Bericht wurden Kopien nach `report/figures/generated/` übernommen.

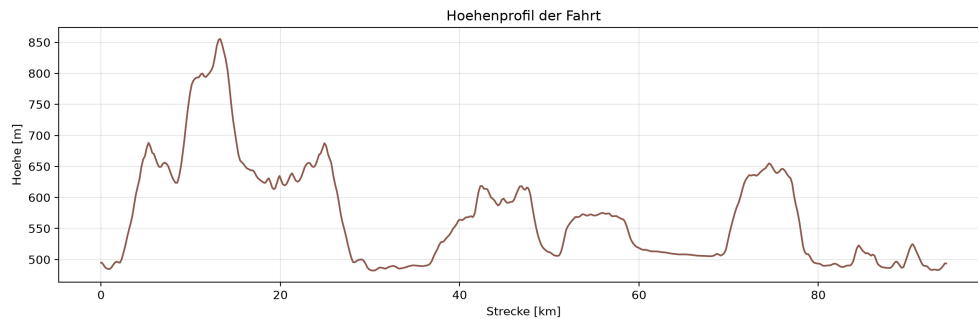


Abbildung 11.1: Höhenprofil der betrachteten Route

Abbildung 11.1 zeigt das Höhenprofil der Strecke. Es ist wichtig für die Leistungs- und Akkuabschätzung, da Steigung und Gefälle direkt in die Hangabtriebskraft eingehen.

Abbildung 11.2 zeigt vier übereinander liegende Diagramme mit gemeinsamer Zeitachse: Geschwindigkeit (mit Rohkurve im Hintergrund, falls Glättung aktiv ist), Leistung, Ladezustand sowie Strom und Spannung auf einer geteilten y-Achse. Leistungsspitzen und Stromverlauf folgen dabei direkt der aktiven Geschwindigkeitskurve. Für NMC entsteht die gleiche Grafik analog.

Abbildung 11.3 vergleicht beide Akkus im gleichen Fahrprofil. Da der Akku im Datensatz leer wird, laufen beide Kurven bis auf 0 % herunter.

Abbildung 11.4 zeigt den Filtereffekt: Die geglättete Kurve entfernt schnelle Zacken, ohne die Strecke selbst zu verändern.

11.3 Logdatei und Verzeichnisse

Die Logdatei `output/ebike_simulation.log` spiegelt die Konsolenausgaben mit Zeitstempeln. Alle Verzeichnisse legt das Programm beim Start automatisch an.

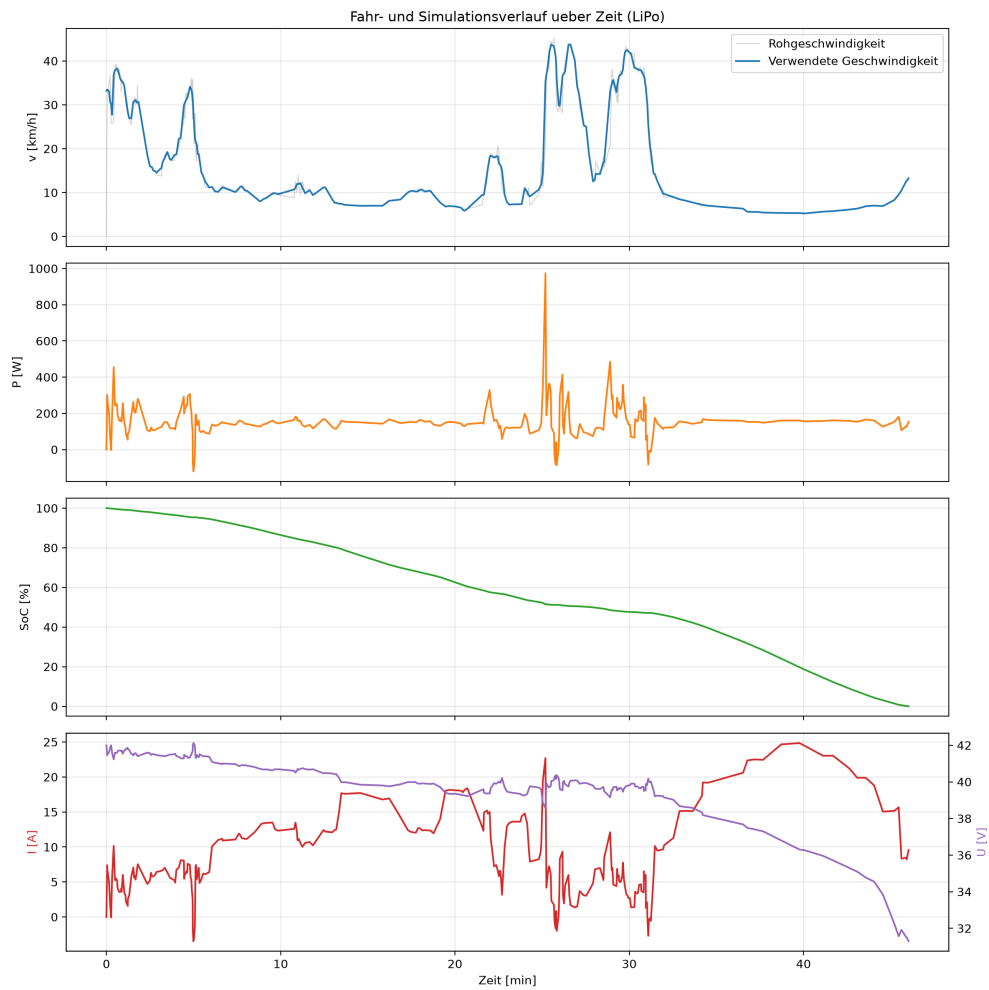


Abbildung 11.2: Zeitverlauf der Simulationsgrößen für den LiPo-Lauf

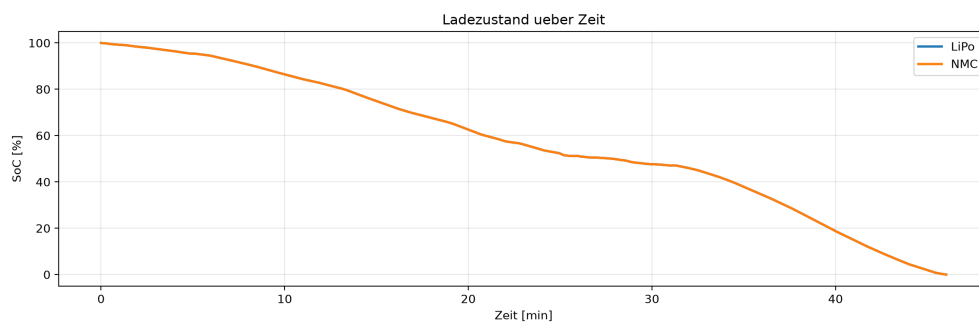


Abbildung 11.3: Vergleich des Ladezustands beider Akkutypen

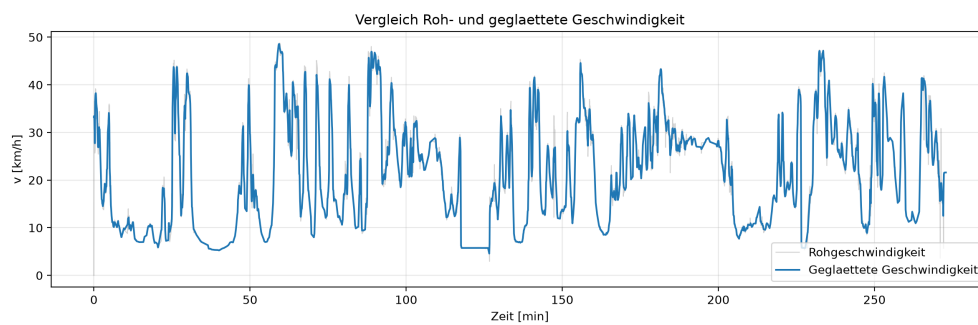


Abbildung 11.4: Vergleich von Roh- und geglätteter Geschwindigkeit

Kapitel 12

Reverse Geocoding, Wetterdaten und Web-Anwendung

Dieses Kapitel beschreibt die im aktuellen Code vorhandenen Erweiterungen: Reverse Geocoding, Wetterdaten, die GeoPandas-Kartenvariante und die Flask-Web-Anwendung mit Parameterstudien.

12.1 Reverse Geocoding: von Koordinaten zu Adressen

`GPSTrack.orte_ermitteln()` wandelt einzelne GPS-Punkte in lesbare Adressen um. Dafür nutzt die Methode den Dienst Nominatim von OpenStreetMap über die Python-Bibliothek `geopy`. Da Nominatim maximal eine Anfrage pro Sekunde erlaubt und keine Massenabfragen, wird nicht jeder der teils über 2000 GPS-Punkte abgefragt. Stattdessen wählt die Methode Start, Ziel und eine konfigurierbare Anzahl gleichmäßig verteilter Zwischenpunkte (Standard: 8) aus und fragt nur diese ab. Ein `RateLimiter` erzwingt dabei die Mindestpause zwischen zwei Anfragen.

Für jeden gewählten Punkt liefert `geolocator.reverse((lat, lon))` entweder eine Adresse oder `None`, falls die Koordinate keiner Adresse zugeordnet werden kann, zum Beispiel mitten im Wald. Schlägt eine Anfrage durch einen Netzwerkfehler fehl, wird das abgefangen und geloggt, statt das Programm abstürzen zu lassen. Das Ergebnis – Index, Koordinaten und Adresse je Punkt – wird nur im Arbeitsspeicher als `self.orte` gehalten und nicht auf die Festplatte geschrieben. `karte_erstellen()` liest `self.orte` automatisch aus und zeigt die Adressen in den Marker-Popups an (siehe Kapitel 11).

12.2 Wetterdaten: woher die Werte kommen

`wetterdaten.py` fragt reale Wetterdaten über die kostenlose Open-Meteo-API ab (<https://open-meteo.com/>, kein API-Key nötig). Um die Anzahl der Anfragen klein zu halten, werden nicht alle GPS-Punkte abgefragt, sondern genau dieselben Punkte, die zuvor per Reverse Geocoding bestimmt wurden – Koordinaten und Adresse liegen dafür ohnehin schon vor.

Alle gewählten Orte werden in einer einzigen HTTP-Anfrage abgefragt, indem ihre Koordinaten kommagetrennt übergeben werden. Je nachdem, wie weit der Aufzeichnungszeitpunkt in der Vergangenheit liegt, wählt das Programm automatisch die passende Open-Meteo-Schnittstelle: die Archiv-API für länger zurückliegende Zeitpunkte (mehr als 5 Tage) oder die Forecast-API für aktuelle bzw. nahe Zeitpunkte. Die Antwort enthält für jeden Ort einen stündlichen Verlauf; das Programm wählt daraus den Stundenwert, der dem tatsächlichen Aufzeichnungszeitpunkt am nächsten liegt. Abgefragt werden Temperatur, Windgeschwindigkeit, Niederschlag und Luftfeuchtigkeit. Schlägt die Abfrage fehl, bleibt die Simulation lauffähig und die betroffenen Werte werden als fehlend behandelt.

12.3 Statische GIS-Karten mit GeoPandas

Neben der interaktiven `folium`-Karte (Kapitel 11) bietet `GPSTrack` mit `karte_erstellen_geopandas()` eine zweite, inhaltlich ähnliche Kartendarstellung als statisches PNG. Der Aufbau folgt derselben Logik wie bei `karte_erstellen()`: Die Strecke wird in einzelne Liniensegmente zwischen je zwei aufeinanderfolgenden GPS-Punkten zerlegt, jedes Segment bekommt den Messwert des zugehörigen Punktes zugeordnet, und alle Segmente werden als `geopandas.GeoDataFrame` nach einer Farbskala eingefärbt geplottet. Der Unterschied liegt im Format: Statt einer interaktiven Leaflet-Karte entsteht ein Vektordatensatz (Koordinatensystem EPSG:4326, für die Darstellung nach Web-Mercator umprojiziert), der sich zum Beispiel als GeoJSON oder Shapefile exportieren und in echter GIS-Software wie QGIS weiterverwenden lässt. Ein optionaler OpenStreetMap-Hintergrund über `contextily` wird nur geladen, wenn Internetzugriff verfügbar ist; schlägt das fehl, entsteht die Karte automatisch ohne Hintergrund statt mit einem Fehlerabbruch.

Für Fälle, in denen nur der reine Streckenverlauf interessiert – ohne Einfärbung nach einem Messwert –, gibt es zusätzlich `karte_erstellen_geopandas_einfach()`: Sie behandelt die gesamte Strecke als ein einziges `LineString`-Objekt statt vieler Einzelsegmente, was schneller ist und eine kleinere Ausgabedatei erzeugt.

12.4 Web-Anwendung und automatische Parameterstudien

Neben dem Kommandozeilenaufruf über `main.py` bietet das Projekt eine kleine Flask-Web-Anwendung (`app.py`, `webapp/`), die dieselbe Simulationslogik über eine Browseroberfläche zugänglich macht. Auf der Startseite lassen sich Fahrzeug-, Glättungs- und Akkuparameter über ein Formular einstellen und optional eine eigene GPS-CSV-Datei hochladen; die Route `/simulate` führt die Simulation aus und zeigt Kennzahlen, Plots und Karten auf einer Ergebnisseite an.

Die Route `/parameterstudie` automatisiert Ein-Faktor-Parameterstudien: Ein einzelner Fahrzeugparameter (zum Beispiel Masse, `cW-A` oder Raddurchmesser) wird über einen gewählten Wertebereich in mehreren Schritten variiert, und die Simulation läuft für jeden Wert erneut. `run_parameter_study()` in `webapp/services/parameter_study_service.py` sammelt die Kennzahlen jedes Laufs in ei-

ner Tabelle, darunter Gesamtstrecke der Route, tatsächlich simulierte Distanz, Abschlussstatus, Abschlussgrad, End-SoC, minimale Spannung und Maximalleistung. Die Web-Oberfläche zeigt diese Tabelle direkt an und erlaubt den CSV-Export. Eine automatische Textinterpretation der Ergebnisse ist im aktuellen Code nicht enthalten; die Bewertung erfolgt anhand der Tabelle und der Diagramme.

Kapitel 13

Tests und Validierung

Das Projekt enthält drei Testdateien. Sie prüfen Konfiguration und Verhalten der Geschwindigkeitsglättung, die Integration mit GPSTrack und dem Simulator sowie das Verhalten der Web-Anbindung. Statt des realen, über 2000 Punkte langen Datensatzes bauen die Tests kleine, synthetische GPS-CSV-Dateien mit genau bekannten Zeit- und Distanzwerten. Damit lässt sich für jeden Testfall gezielt und deterministisch prüfen, ob die Glättung, die Routenverarbeitung und die Web-Rückgaben das erwartete Verhalten zeigen.

13.1 Ausgeführter Testbefehl

```
1 python -m pytest -q
```

13.2 Ergebnis

Die Test-Suite hat 21 Einzeltests. Abgedeckt sind Konfigurationsladung, Aktivierung und Deaktivierung der Glättung, konstante Geschwindigkeit, Ausreisser, kurze Intervalle, grosse Zeitlücken, unregelmäßige Zeitabstände, die Kompatibilität mit dem Simulator, die Web-Simulation, die Geocoding- und Wetterzweige sowie die Erkennung vollständiger und unvollständiger Routen.

13.3 Interpretation

Die Tests prüfen nicht die volle numerische Genauigkeit, sondern das erwartete Verhalten an den kritischen Stellen. Sie sichern vor allem ab, dass die Glättung keine pauschale Begrenzung einbaut, der Simulator die aktiven Spalten korrekt weiterverarbeitet und die Web-Anbindung die Ergebnisse aus dem gemeinsamen Service übernimmt.

Tabelle 13.1: Abgedeckte Testbereiche

Testbereich	Geprüfte Aussage
Konfigurationsladen	YAML-Werte werden eingelesen, Defaults greifen und ungültige Werte führen zu Fehlern.
Aktivierte und deaktiverte Glättung	Die aktiven Spalten folgen entweder den Roh- oder den geglätteten Werten.
Konstante Geschwindigkeit	Die Beschleunigung bleibt nahe null.
Einzelner Ausreisser	Ein grosser Geschwindigkeitssprung wird als Filterstützstelle ausgeschlossen.
Kurzes Intervall	Sehr kleine Zeitabstände werden nicht als gültige Stützstellen verwendet.
Grosse Zeitlücke	Neue Segmente beginnen an den richtigen Stellen, ohne über die Lücke zu interpolieren.
Unregelmässige Intervalle	Der zeitabhängige exponentielle Faktor reagiert auf unterschiedliche Zeitabstände.
Simulator-Kompatibilität	EBikeSimulator verarbeitet das Track-DataFrame mit den aktiven Spalten korrekt.
Web-Routing	Die Web-Route übergibt die erwarteten Formularwerte an den gemeinsamen Simulationsservice.
Orts- und Wetterdaten	Die Simulation arbeitet ohne optionale Abfragen weiter, auch wenn sie deaktiviert oder fehlerhaft sind.
Routenabschluss	Vollständige und unvollständige Fahrten werden anhand der simulierten Punkte korrekt erkannt.

Kapitel 14

Ergebnisse

Die folgenden Kennzahlen stammen aus dem aktuellen Datensatz. Die Streckenkennwerte sind bei beiden Glättungsläufen gleich, da sie nur von den GPS-Daten abhängen; Unterschiede zeigen sich nur bei Geschwindigkeit, Beschleunigung und den elektrischen Größen.

14.1 Streckenkennwerte

Tabelle 14.1: Ermittelte Kennzahlen des aktuellen Datensatzes

Kennwert	Wert	Kommentar
GPS-Punkte	2284	Anzahl der eingelesenen Zeilen der CSV-Datei, also der Stützstellen der gesamten Fahrt.
Gesamtdauer	16372.391 s	<code>gesamtzeit_s()</code> : Zeitspanne zwischen erstem und letztem Zeitstempel (≈ 4.55 h).
Gesamtstrecke	94.267 km	<code>gesamtstrecke_km()</code> : Summe aller Haversine-Distanzsegmente <code>distanz_m</code> .
Höhenmeter aufwärts	1095.895 m	<code>hoehenmeter_anstieg()</code> : Summe aller positiven Höhendifferenzen Δh_i .
Höhenmeter abwärts	1097.146 m	<code>hoehenmeter_abstieg()</code> : Summe aller negativen Höhendifferenzen (Betrag).
Durchschnittsgeschwindigkeit	20.728 km/h	Gesamtstrecke geteilt durch Gesamtdauer, unabhängig von der Glättung.
Maximale Rohgeschwindigkeit	48.772 km/h	Maximum der Spalte <code>geschwindigkeit_roh_ms</code> , also vor jeder Filterung.
Maximale geglättete Geschwindigkeit	48.584 km/h	Maximum der Spalte <code>geschwindigkeit_geglaettet_ms</code> nach Median- und Exponentialfilter.
Maximale Rohbeschleunigung	2.982 m/s ²	Größter positiver Wert der Spalte <code>beschleunigung_roh_ms2</code> .
Minimale Rohbeschleunigung	-3.790 m/s ²	Stärkste Verzögerung (negativer Extremwert) vor der Glättung.

14.2 Vergleich der Glättung

Die Strecke bleibt gleich, aber die Glättung senkt die Spitzen deutlich. Ohne Glättung führt ein kurzer Geschwindigkeitssprung zu einer hohen Lastspitze; mit Glättung wird der Verlauf ruhiger, während Strecke und mittlere Geschwindigkeit fast gleich bleiben.

14.3 Interpretation

Beide Akkus entladen sich im Datensatz vollständig – der Lastbedarf der Strecke ist einfach zu hoch. Das heisst nicht, dass beide Akkus gleich reagieren: Die NMC-Kurve liegt bei gleichem SoC etwas niedriger in der Spannung als die LiPo-Kurve.

Der Hauptnutzen der Glättung zeigt sich in den Spitzenwerten: Sie senkt die maximale Leistung auf rund 974 W statt 2516 W und den maximalen Strom auf etwa 24,8 A statt 61,6 A. So wird der Einfluss von GPS-Rauschen direkt sichtbar.

Tabelle 14.2: Vergleich ohne und mit Glättung

Kennwert	Glättung aus	Glättung an	Kommentar
Aktive Geschwindigkeitsspitze	48.772 km/h	48.584 km/h	Maximum der jeweils aktiven Spalte <code>geschwindigkeit_ms</code> ; kaum Unterschied, da die Glättung nur kurze Spitzen dämpft.
Aktive Beschleunigungsspitze	2.982 m/s ²	0.583 m/s ²	Zeigt den stärksten Glättungseffekt: Ausreisser in der Beschleunigung werden deutlich reduziert.
Maximale mechanische Leistung	2515.913 W	974.146 W	<code>maximalleistung_W()</code> ; folgt direkt aus $P = Fv$, da F mit der Beschleunigungskraft ma skaliert.
Maximaler Motorstrom	61.628 A	24.843 A	<code>get_current_draw()</code> : $I = M/k_m$; sinkt proportional zur geringeren Drehmomentspitze.
End-SoC LiPo	0.0 %	0.0 %	<code>endladezustand_prozent()</code> ; beide Läufe entladen den Akku vollständig, da die Gesamtenergie der Strecke ausschlaggebend ist.
End-SoC NMC	0.0 %	0.0 %	Gleiches Verhalten wie beim LiPo-Pack, da beide Akkus dieselbe Kapazität und dasselbe Lastprofil sehen.
Minimale Spannung LiPo	31.278 V	31.292 V	Nahe an $V_{\min}=32$ V; die Simulation bricht ab, sobald <code>is_empty()</code> erfüllt ist.
Minimale Spannung NMC	31.115 V	31.097 V	Etwas niedriger als LiPo, da die NMC-OCV-Kennlinie bei gleichem SoC eine geringere Spannung liefert.
Thea Welk, Emil Kupfer, Franciszek Wozniak			38
Bremswiderstandsenergie	0.000 Wh	0.000 Wh	<code>bremswiderstand_energie_Wh()</code> ; bleibt null, weil im Datensatz nie mehr Ladestrom als Kapazität anfällt (kein Überladen).

Kapitel 15

Grenzen und Ausblick

Das aktuelle Modell ist bewusst schlank gehalten. Es beschreibt die im Quellcode umgesetzten Zusammenhänge nachvollziehbar, ersetzt aber keine vollständige Fahrzeugsimulation.

15.1 Aktuell implementiert

Implementiert sind GPS-basierte Kinematik, Fahrzeugkräfte, Motorstrom, Akku-SoC, Temperaturkorrektur, Luftdichteabschätzung, optionale Orts- und Wetterdaten sowie die Behandlung eines Bremswiderstandsfalls bei zu hohem Ladestrom. Die Web-Anwendung und die Kommandozeile verwenden denselben Simulationsservice, aber unterschiedliche Bedienoberflächen.

15.2 Modellannahmen

Mehrere Vereinfachungen sind für die aktuelle Version wesentlich. Die Fahrerleistung wird nicht separat modelliert. Der Motor wird nur über eine konstante Motorkonstante beschrieben. Der Luftwiderstand nutzt eine einfache quadratische Formel. Die thermische Akkuphysik wird über lineare Korrekturfaktoren angenähert. GPS-Höhe und GPS-Position werden direkt als Eingangsrealität akzeptiert, obwohl beide Messgrößen verrauscht sein können. Orts- und Wetterdaten hängen von externen Diensten ab und werden bei Fehlern bewusst nicht als harter Abbruch behandelt.

15.3 Mögliche Erweiterungen

Naheliegende Erweiterungen wären ein Windmodell, ein detaillierteres Motorwirkungsgradmodell, eine echte Rekuperationslogik mit begrenzter Ladeleistung, eine mechanische Fahrerleistung, eine Validierung gegen reale Sensordaten sowie eine ausgereifere Filterparametrierung. Auch die Akkumodelle könnten um temperatur- und lastabhängige Kennlinien erweitert werden. Eine serverseitige PDF-Erzeugung für Simulationsergebnisse ist im aktuellen Stand nicht Bestandteil des Projekts.

15.4 Einordnung

Der Wert des Projekts liegt nicht in maximaler Komplexität, sondern in der sauberen Kette von Daten zu physikalischer Last und anschließender Akku-Simulation. Für den vorhandenen Datensatz ist diese Kette bereits schlüssig nachvollziehbar und reproduzierbar.

Kapitel 16

Fazit

Das Abschlussprojekt zeigt, wie sich eine echte GPS-Strecke zu einer reproduzierbaren E-Bike-Energiesimulation ausbauen lässt. Die Software trennt Datenverarbeitung, Fahrzeugphysik, Motorlogik, Akkuverhalten, Visualisierung und Web-Oberfläche klar voneinander – bis hin zu Karten, Plots und Logs. Der Bericht selbst wird mit `latexmk` automatisiert erstellt **latexmk_docs**.

Besonders wichtig ist die Polymorphie der Akkumodelle: LiPo und NMC nutzen dieselbe Pipeline, unterscheiden sich aber in Kennlinie und Innenwiderstand. Zusammen mit der optionalen Glättung zeigt sich so gut, wie empfindlich Leistung und Strom auf GPS-Rauschen reagieren. Die Web-Anwendung verwendet dieselbe Simulationslogik wie die Kommandozeile, wodurch beide Einstiegspunkte denselben fachlichen Stand abbilden.

Im aktuellen Datensatz ist die Fahrt so anspruchsvoll, dass beide Akkus vor Streckenende leer sind. Genau das macht den Bericht wertvoll: Die Simulation liefert nicht nur Bilder, sondern eine klare Aussage über das Zusammenspiel von Strecke, Modell und Energiespeicher. Die Grenzen des aktuellen Standes liegen vor allem in den vereinfachten Modellannahmen und den optionalen externen Diensten für Orts- und Wetterdaten.